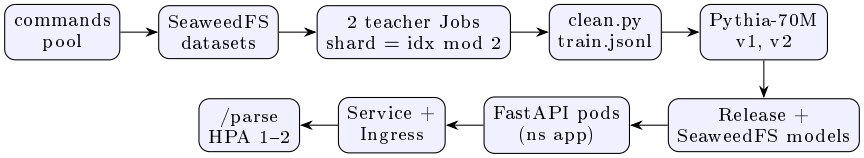
CS3081 Mini Project — Technical Report

Aniket Raj, roll no. 230103030, GitHub `iamankoo`, `github.com/iamankoo/Cloud-Assignment`

Evidence policy. This is an independent but *incomplete* submission. **Implemented/tested** = exists and was run locally (14 unit tests, pyflakes, own NACL evaluator, subnet script; logs in `results/`). **Method** = procedure described, *not* executed. [PLACEHOLDER] = no measurement exists. No training, labelling, Docker build, cluster run, CI run or load test was performed. Expected outcomes are theory, not results.

1. Introduction and architecture

A 3.8B-parameter teacher (Phi-3.5 Mini via Ollama) labels 3,000 commands once, offline; a 70M student (Pythia-70M) imitates those labels (sequence-level knowledge distillation); only the student is served.



Task	Status	Evidence / gap
1	code written, not run	worker, cleaning (unit-tested on toy data), Job manifest; no kit data, Ollama or cluster
2	code written, not run	training script and config; no labelled data, no training
3	implemented, tested (stub)	5 API tests; image not built, no 50-request test
4	manifest only	YAML fields tested; never applied
5	implemented, tested	subnet script and own evaluator 11/11; instructor's `nacl_sim.py` not run
6	partial	workflow, Makefile (`make up/down`, not run); no CI run; no Release gate
7	code written, not run	Locust file, cost script; nothing deployed, no measurements

Model artifacts. The GitHub Release `reference-nanoparse-v1.0` holds two Pythia-70M archives that are *externally sourced reference artifacts* (rahul-singh92/NanoParse release v1.0, author Rahul Singh Jadoun), re-uploaded unmodified. They were not trained or evaluated in this project and are not reported as my results (see `docs/RELEASE_METADATA.md`). Paths in this report use the GitHub repository layout; the submission ZIP maps them as listed in its `README.md`.

2. Task 1 — Object storage and parallel teacher labelling (method)

Design. SeaweedFS (S3 gateway) in namespace `data`, buckets `datasets` and `models`. An Indexed Job (`completions=parallelism=` in `pipeline`; worker i labels records with $n \bmod 2 = i$. Shards are complete (every n has one residue), disjoint, and balanced ($1500 + 1500$). **Cleaning** keeps a reply only if: valid JSON; exactly keys `intent,date,time,place`; intent in the eight allowed; each non-null slot occurs in the command; not a duplicate. Check: $N_{in} = N_{kept} + \sum N_{rejected}$; publish SHA-256 of `train.jsonl`. **Measure:** per-shard counts [PLACEHOLDER]; Job duration [PLACEHOLDER]; rejection counts per rule [PLACEHOLDER]; kept [PLACEHOLDER]; checksum [PLACEHOLDER]. **Limitation:** both workers share one host CPU for Ollama, so speed-up may be small.

3. Task 2 — Student training (method)

Sequence-level KD (Kim and Rush, 2016): train on the teacher's best sequence y^* with cross-entropy,

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log p_\theta(y_t^* \mid x, y_{<t}^*) \,,$$

prompt tokens masked. Input format `Command: <text>\nJSON: <json>` — identical to the service prompt. Pythia-70M, first 1,500 cleaned records, AdamW $\approx 5 \times 10^{-5}$, batch 8 (188 steps/epoch), 3–4 epochs, seed fixed, all values in `training/config.yaml`; `make train`, `make train SUBSET=50`. v1 and v2 differ by a recorded setting; both get SHA-256 and a GitHub Release asset; weights are never committed.

Model	JSON valid	Intent	Date	Time	Place
Teacher	[PLACEHOLDER]	[PLACEHOLDER]	[PLACEHOLDER]	[PLACEHOLDER]	[PLACEHOLDER]
Untrained Pythia-70M	[PLACEHOLDER]	[PLACEHOLDER]	[PLACEHOLDER]	[PLACEHOLDER]	[PLACEHOLDER]
Student v1	[PLACEHOLDER]	[PLACEHOLDER]	[PLACEHOLDER]	[PLACEHOLDER]	[PLACEHOLDER]
Student v2	[PLACEHOLDER]	[PLACEHOLDER]	[PLACEHOLDER]	[PLACEHOLDER]	[PLACEHOLDER]

Expected (theory): the untrained model scores near 0% validity; the trained student improves validity sharply but cannot exceed its teacher's slot accuracy. Loss per epoch: [PLACEHOLDER].

4. Task 3 — FastAPI inference service and Docker

`POST /parse`, `GET /healthz`, `GET /readyz` (503 until the model is loaded). Output pipeline: `raw` → `json.loads` → `jsonschema.validate`; failure → HTTP 500 `invalid_model_output`; `FAULT_INJECT=1` substitutes `not json`. Success sets `X-Model-Version` and `X-Served-By`. Dockerfile: `python:3.11-slim`, CPU-only torch 2.4.1, pinned dependencies, no weights, limit ≤2 GB. **Tested:** 5 stub-model tests pass (success+headers, bad intent, extra key, fault injection, health/ready). **Not done:** image build and size [PLACEHOLDER]; 50 sequential requests at 1 CPU/1 GiB, peak memory [PLACEHOLDER].

5. Task 4 — Kubernetes

Namespace `app`: Deployment (2 replicas; requests 250m/512Mi; limits 1 CPU/1Gi; liveness `/healthz`; readiness `/readyz`; RollingUpdate maxUnavailable 0, maxSurge 1) → Service :80 → Ingress (Traefik); HPA min 1, max 2, CPU 70%. HPA rule:

$$\text{desired} = \left\lceil \text{current} \times \frac{\text{utilisation}}{\text{target}} \right\rceil, \quad \text{e.g. } \lceil 2 \cdot 140/70 \rceil = 4 \to \text{clipped to } 2.$$

Procedures: replica distribution via `X-Served-By`; rollout by `kubectl set env` (v1→v2) under load then `rollout undo`; pod deletion under load; three cold starts. Only the YAML fields were tested. Results: failed requests during rollout (target ≤ 0.5%) [PLACEHOLDER]; version after rollback [PLACEHOLDER]; recovery time [PLACEHOLDER]; cold starts [PLACEHOLDER]; HPA events [PLACEHOLDER]. **Limitation:** single host, no node-failure tolerance.

6. Task 5 — Subnetting and NACL (implemented, tested)

Prefix $p = 32 - \lceil \log_2(h+2) \rceil$; blocks placed contiguously, largest first.

Subnet	h	Network	Usable range	Broadcast	Usable
app	1000	10.20.0.0/22	.0.1–.3.254	10.20.3.255	1022
public	250	10.20.4.0/24	.4.1–.4.254	10.20.4.255	254
data	100	10.20.5.0/25	.5.1–.5.126	10.20.5.127	126
management	20	10.20.5.128/27	.5.129–.5.158	10.20.5.159	30

Example: $\log_2 1002 = 9.97 \Rightarrow p = 22$, 1022 usable. **NACL algorithm:** sort rules ascending; first rule matching CIDR (source inbound, destination outbound) and protocol/port decides; none ⇒ deny. Inbound: 100 deny 198.51.100.0/24 (all); 110 tcp 80; 120 tcp 443; 130 tcp 22 from 10.20.5.128/27; 140 tcp 1024–65535 from 10.20.0.0/22. Outbound: 100 tcp 8000 to 10.20.0.0/22; 110 tcp 1024–65535 to 0.0.0.0/0. **Why replies need rules:** a NACL is stateless; a reply is a new packet judged alone. **Why order matters:** rule 100 precedes 110/120, otherwise 198.51.100.0/24 would be allowed on 80/443. **Side effect:** rule 110 also permits outbound TCP 8000 to anywhere. **Tested:** own evaluator 11/11 packets; the instructor's six-packet `nacl_sim.py` was not run.

7. Task 6 — Reproducibility, CI/CD, security

Written: Makefile (`test, lint, nacl, subnets`); `.github/workflows/ci.yml` (pyflakes, pytest, gitleaks); pinned dependencies; `.gitignore` excluding weights and `.env`; `AI_USE.md`. **Not done:** any CI run, model evaluation from a Release, the 95% validity gate, the deliberately failing run (link [PLACEHOLDER]), `make up`. Gate logic:

`score.py -min-validity 95` exits non-zero below 95%. Security: secrets only in Kubernetes/CI secrets; shared S3 identity is a known limitation.

8. Task 7 — Performance and cost (method)

Locust, 4 users, 2 minutes through the Ingress; failure = not HTTP 200 within 30 s. Metrics: p50/p95 latency, throughput = requests/s, error rate = failures/requests, `kubectl top` peak CPU/memory, cold-start median. Cost: $C_h = r\,(c\,P_{cpu} + m\,P_{mem})$ and

$$\text{cost per 1000 req} = \frac{C_h}{3600 \cdot \text{throughput}} \times 1000,$$

assuming billing at limits (an assumption). Error budget $= (1 - \text{SLO}) \times N$.

Metric	Value
p50 / p95 latency	[PLACEHOLDER]
Throughput	[PLACEHOLDER]
Error rate	[PLACEHOLDER]
Peak CPU / memory	[PLACEHOLDER]
Cold-start median	[PLACEHOLDER]
Cost per 1000 requests	[PLACEHOLDER]
SLO verdict	[PLACEHOLDER]

Review table (six rows, evidence/limitation/improvement): requires real numbers [PLACEHOLDER].

9. Limitations and references

No machine with Docker, k3d or Ollama and no student_kit were available; Tasks 1, 2, 7 and all cluster/CI behaviour are unverified. References: Kim and Rush, *Sequence-Level Knowledge Distillation*, 2016; Hinton et al., *Distilling the Knowledge in a Neural Network*, 2015; Biderman et al., *Pythia*, 2023; Kubernetes, k3d, FastAPI, Locust, AWS VPC (NACL) and Google SRE Book documentation.